FastAPI Campaign Management Backend – Assignment Solution

Project Summary

This backend system was developed as a solution for the Campaign Management assignment problem. It provides a FastAPI-based service to manage and run discount campaigns for customers, with full CRUD functionality, logging, Dockerization, and an SQLite database. The application enables discounts to be configured for carts or delivery, limits discount usage per customer, and supports campaign duration and budget constraints. All business rules described in the problem statement are implemented, providing a scalable and maintainable backend solution.

Overview

This backend provides REST APIs for managing discount campaigns and customers. It is built with FastAPI, SQLAlchemy, SQLite, and Loguru, with Docker for easy deployment.

Features

- CRUD for campaigns and customers
- Assign discounts to cart or delivery
- Set campaign duration and budget
- Restrict discount usage per customer per day
- Target specific customers
- Fetch available campaigns for a customer
- API documentation via Swagger UI
- Logging with Loguru
- Organized codebase (routes, dependencies, schemas)
- Dockerized for production

Tech Stack

- Python 3.12
- FastAPI
- SQLAlchemy
- SQLite
- Loguru
- Docker

Project Structure

```
backend/
 ├── api/            # Routes, dependencies, schemas
 ├── app/            # Log files
 ├── core/           # Config, logger
```

```
├── db/              # Database models, connection
├── main.py          # App entrypoint
├── requirements.txt    # Python dependencies
├── Dockerfile       # Docker build file
├── entrypoint.sh      # Docker entrypoint script
└── campaigns.db       # SQLite database file
```

Getting Started

Local Development:

Create virtual environment before this
1. Install dependencies:
   pip install -r requirements.txt
2. Run the app:
   python main.py
3. Open API docs:
   http://localhost:8000/docs

Docker:
1. Build the image:
   docker build -t fastapi-campaigns .
2. Run the container:
   docker run -p 8000:8000 fastapi-campaigns start-api-server

Main API Endpoints

Campaigns:
- POST /campaigns/ – Create campaign
- GET /campaigns/ – List campaigns
- GET /campaigns/available – Get available campaigns for a customer
- GET /campaigns/{campaign_id} – Get campaign by ID
- PATCH /campaigns/{campaign_id} – Update campaign (partial)
- DELETE /campaigns/{campaign_id} – Delete campaign

Customers:
- POST /customers/ – Create customer
- GET /customers/ – List customers
- GET /customers/{customer_id} – Get customer by ID
- PUT /customers/{customer_id} – Update customer
- DELETE /customers/{customer_id} – Delete customer

Health:
- GET /health/ – Health check

Environment & Logging

- Optional .env file for configuration
- Logs stored in app/ folder by date
- Loguru used for daily rotated logs

Extending

- Add new routes in api/routes/
- Add business logic in api/dependencies/
- Extend entrypoint.sh for additional services
- Upgrade to PostgreSQL/MySQL for production